University of Galway
Electronic & Computer Engineering Department

GDPR Facial Anonymiser

May 2026

Department of Electronic & Computer Engineering
National University of Ireland Galway

PROJECT REPORT (Year-5)

GDPR Facial Anonymiser

Group members:
Séamus Knightly – 21450934

Date of submission: 10/5/2026

Supervisor: Prof. Peter Corcoran

Co-Assessor: Dr. Soumyajyoti Maji

Abstract

Video surveillance systems routinely capture facial images of individuals who did not consent to identification, creating tension with the requirements of the EU General Data Protection Regulation (GDPR). Existing face anonymisation approaches occupy two extremes: simple Gaussian blur, which is fast but partially reversible and destroys useful spatial information, and GAN, or diffusion,based identity synthesis, which can produce high-quality results but requires GPU inference incompatible with embedded deployment.

This project designs and implements a real-time face anonymisation pipeline targeting the Raspberry Pi 5. The system uses MediaPipe BlazeFace to detect faces and extract facial keypoints, constructs a composite mask using landmarks, covering the eyes, nose, and mouth, and fills the masked region using a custom trained TFLite inpainting model. The model is a lightweight Mobile U-Net (˜3.2 million parameters, 128×128 input) trained on CelebA and VGGface2 using a combined masked L1, VGG16 perceptual, and FaceNet re-identification repulsion loss. Two deployment TFLite variants are produced: float16 and INT8 quantisation-aware training (QAT). The pipeline is served as an MJPEG stream from an HTTP server with a web dashboard for runtime configuration.

Evaluation on a 1,000-image test set shows that the float16 model achieves 98.0% face detection survival after anonymisation, with a face-region SSIM of 0.809 and mean absolute error of 10.25 pixels, at a desktop end-to-end latency of 63 ms. On the Raspberry Pi 5, end-to-end latency is 71 ms for a single face (40 ms inference), sufficient for approximately 14 frames per second. A gallery-based re-identification attack using ArcFace shows the float16 model reduces rank-1 re-identification rate from 73.8% (Gaussian blur baseline) to 30.9%, with a substitute identity effect whereby the anonymised face is more similar to a different gallery identity than to the original subject. The INT8 model reduces inference to 19 ms on the Pi and, after resolving three implementation bugs in the quantisation pipeline, achieves 98.4% detection survival, comparable to the float16 model. A GDPR assessment concludes that the output constitutes pseudonymisation rather than full anonymisation, as outer face biometrics and contextual cues are preserved by design.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Background

1.1 Problem Statement and Motivation

Video surveillance is widely deployed in public and semi-public spaces: retail environments, transport hubs, workplaces, and public streets. These systems capture continuous footage containing the faces of individuals who have not consented to being recorded, or who have consented only to the recording of non-identifying information (for example, crowd density or queue length). Under the European Union's General Data Protection Regulation (GDPR) [3], facial images constitute personal data, and their processing requires a lawful basis. One approach to establishing such a basis is to anonymise faces at the point of capture, so that the downstream video stream does not contain identifying information.

The naive solution currently includes the blurring or pixelating detected faces, which is computationally simple but comes with practical drawbacks. It destroys facial attributes that remain useful for non-identifying analysis (head pose, gaze direction, expression, mouth state), and it is partially reversible; dedicated deblurring networks can recover face structure from known blur kernels [4]. More sophisticated approaches exist in the research literature, ranging from GAN-based identity synthesis to diffusion-model anonymisation, but these methods require GPU/NPU inference and are incompatible with real-time deployment on the embedded hardware typically used in surveillance systems.

This project addresses the gap between these extremes: a face anonymisation system that suppresses identifying inner-face features using a trained inpainting model, runs in real time on a Raspberry Pi 5, and preserves non-identifying spatial information useful for downstream tasks.

1.2 GDPR Context

GDPR Article 4(1) defines personal data as any information relating to an identified or identifiable natural person. Facial images fall squarely within this definition. The regulation distinguishes between two degrees of de-identification, including anonymisation, in which re-identification is impossible or would require unreasonable effort, and pseudonymisation (Article 4(5)), in which a direct identifier is replaced by a pseudonym but re-identification remains possible with additional information. Anonymous data falls outside the regulation entirely, pseudonymous data remains in scope.

The European Data Protection Board's 2025 guidelines [5] clarify that most practical face de-identification techniques produce pseudonymous rather than anonymous data, outer-face biometrics, contextual cues, and gait information typically remain accessible. The goal of this project is therefore not to claim full anonymisation, which the system does not achieve, and might

not feasibly be able to achieve with the stated goals, and instead to implement a privacy-enhancing technology that materially reduces identification risk while remaining deployable on resource constrained hardware.

1.3 Scope and Assumptions

The system anonymises only the **inner face region**: the eyes, nose, and mouth. This scope was chosen because these features carry the most weight in automated face recognition, while the outer face (jawline, hair, ears) and body carry useful non-identifying spatial information.

The following facial attributes are explicitly preserved by design:

- Face presence and bounding box (the face is still detectable after anonymisation)
- Head pose and gaze direction (estimated from outer-face geometry)
- Face scale (proportional to distance from camera)
- Approximate expression (mouth open/closed state is partially preserved)

The following are out of scope and not suppressed:

- Soft biometrics: apparent age, gender, and ethnicity cues visible in the outer face
- Hair, ears, and jawline geometry
- Body, clothing, and contextual frame content

All inference runs locally on the capture device. No unprocessed frames are transmitted to any remote service.

1.4 Summary of Approach and Key Results

The system uses MediaPipe BlazeFace [6] to detect faces and extract six facial keypoints per face, of which five are used for mask placement (the two ear tragions are discarded). A composite mask is constructed from three landmark-guided ellipses covering the eyes, nose, and mouth region. The masked pixels are replaced with uniform noise and the four-channel input (RGB + binary mask) is passed to a lightweight Mobile U-Net inpainting model running as a TFLite flatbuffer. The model's output is composited back into the original frame with feathered alpha blending. The system is served as an MJPEG stream from a Raspberry Pi 5, with a web dashboard for runtime configuration.

The inpainting model was trained on 202,599 CelebA[2] face crops and a supplementary VGGface2[1] dataset (1.2 Million Face Crops), using a four-term loss combining masked L1, context L1, VGG16 perceptual loss, and a FaceNet re-identification repulsion term. Two deployment TFLite variants were produced: float16 and INT8 QAT.

Evaluation on a 1,000-image test set showed that the float16 model achieves 98.0% detection survival, SSIM of 0.928 on the full frame, and 0.809 on the face bounding box, with a desktop end-to-end latency of 63 ms (mean) and 87 ms at the 95th percentile. On a Raspberry Pi 5, total latency is 71 ms for a single face (40 ms inference). The INT8 model runs inference in 19 ms on the Pi and achieves 98.4% detection survival after three quantisation pipeline bugs were identified and resolved.

1.5 Report Structure

Chapter 2 surveys the background literature on face detection for embedded systems, the landscape of face anonymisation techniques from classical blur to state-of-the-art diffusion models, the legal and technical definitions of the anonymisation/pseudonymisation boundary, and the architectural and quantisation techniques that underpin this project. Chapter 3 describes the system architecture and implementation: the detection pipeline, composite mask design, stabilised inference cache, and streaming server. Chapter 4 covers the model training pipeline: dataset preparation, the Mobile U-Net architecture, the loss function, the augmentation strategy, and the quantisation export paths. Chapter 5 presents evaluation results across all model variants on the 1,000-image test set and on the Raspberry Pi 5, followed by a GDPR assessment and a discussion of failure cases. Chapter 6 summarises achievements, reflects on challenges encountered, and proposes directions for future work.

Chapter 2

Background and Related Work

2.1 Face Detection for Embedded Systems

Face detection is the entry point of any anonymisation pipeline. For real-time operation on embedded hardware, the detector must be both accurate and fast enough to process each frame before the next one arrives.

The dominant approach for real-time mobile face detection is the anchor-based single-shot paradigm, in which the model predicts bounding boxes and confidence score for a fixed set of anchor positions in a single forward pass. BlazeFace [6] is a model from this family, designed specifically for mobile GPU inference. It operates with a 128×128 input resolution, a heavily reduced receptive field, and anchor shapes tuned for face aspect ratios, which allow it to run in sub-millisecond time on mobile hardware. BlazeFace also produces six facial keypoints (left eye, right eye, nose tip, mouth centre, and two ear tragions). Four of these (both eyes, nose, mouth) are used to place the anonymisation mask precisely over the identifying features.

YuNet [7] is a complementary approach optimised for accuracy on smaller and more distant faces, distributed as an ONNX model through the OpenCV Zoo. It uses a different detection backbone and produces a five-point landmark, including the mouth corners, to get the same set as BlazeFace, the midpoint of the two mouth corners is obtained. The two detectors occupy different points on the accuracy-speed curve, making them jointly suitable as a primary/backup pair: BlazeFace is used by default for its speed, with YuNet available when detecting faces at greater range.

The main practical constraint for embedded deployment is that the detector must run fast enough to leave sufficient headroom for the anonymisation model. On a Raspberry Pi 5, both detectors add approximately 11–12 ms to the pipeline; the remaining budget must be allocated to the anonymisation step.

2.2 Face Anonymisation Techniques

The face anonymisation literature spans several generations of technique, from simple signal-processing operations to state-of-the-art generative models.

Classical methods such as Gaussian blur and pixelation are the baseline approach. They are computationally trivial and require no trained model, but they are limited in two important ways. First, they alter the face uniformly, destroying attributes (pose, expression, head shape) that should be preserved. Second, they have been shown to be partially reversible, dedicated deblurring networks can partially recover face structure from blurred images, particularly when

the blur kernel is known or estimable [4]. Furthermore, Dietlmeier et al. [8] demonstrated that blurring faces has only a marginal effect on person re-identification accuracy, suggesting that identity cues outside the face region are often dominant, and that face-blurring alone is insufficient as a privacy guarantee.

GAN-based synthesis became the dominant approach for high-quality anonymisation from the late 2010s onwards. Gafni et al. [9] introduced a real-time capable GAN autoencoder that generates a new identity for a detected face while preserving pose, illumination, and lip articulation. The main idea here is that an attractor-repeller perceptual loss causes the generated identity to be attracted toward realism and repelled from the original identity embedding. Subsequent work extended this to full-body anonymisation (DeepPrivacy2 [10]), landmark-conditional generation (L-GAN [11]), expression-preserving synthesis (GANonymization [12], which targets the specific use case of emotion-recognition datasets), and attention-based identity distraction [13], which suppresses identity by redirecting a face recogniser's attention toward non-identifying facial regions.

DeepPrivacy2 was evaluated as a candidate anonymiser early in this project. Its utility advantage over blur is well-established: Hukkelås and Lindseth [14] demonstrated that realistic GAN-based face anonymisation produces no statistically significant drop in downstream computer vision model performance (instance segmentation and object detection on Cityscapes and BDD100K), whereas Gaussian blur degrades keypoint detection AP by more than 10 percentage points on COCO. Despite this utility advantage, DeepPrivacy2 proved impractical on the target hardware: on a Raspberry Pi 5 the full pipeline ran at sub-one-frame-per-minute, driven primarily by the cost of its face detector.

DeepPrivacy2 uses DSFD (Dual Shot Face Detector [15]), a dual-branch feature pyramid network that extracts face candidates twice, once at coarse feature resolution and again after anchor-level feature enhancement, achieving high accuracy at the cost of being far heavier than a single-shot detector. An attempted remedy was to substitute BlazeFace for DSFD to eliminate the detector bottleneck. This failed for a fundamental reason: DeepPrivacy2's GAN was trained exclusively on crops produced by DSFD, which returns highly consistent, tight bounding boxes reliably encompassing the full face region. BlazeFace outputs are less consistent: depending on face scale, head pose, and camera distance, the returned box may span the full head including hairline, encompass only the inner face from brow to chin, or fall somewhere in between. Feeding these misaligned crops into a DSFD-trained GAN places the face at an unexpected position within the crop, producing incorrectly positioned or partially-anonymised outputs. The conclusion was that a model trained end-to-end on BlazeFace detections as the source, rather than adapted from a DSFD-trained model, was required. This motivated the custom training pipeline described in Chapter 4.

Formal privacy guarantees have been pursued through differential privacy. IdentityDP [16] disentangles identity from attribute features and injects calibrated Laplace noise into the identity vector, providing a provable ϵ-differential privacy bound. While theoretically appealing, the quality-privacy trade-off controlled by ϵ is difficult to tune in practice, and the perceptual quality of DP-perturbed outputs is generally lower than GAN-synthesised alternatives.

Diffusion-based methods represent the current state of the art. Kung et al. [17] demonstrated that a latent diffusion model, conditioned only on a reconstruction loss, can remove identity while preserving pose, gaze, and expression, without requiring facial landmarks or segmentation masks. Liu and Lian [18] further decoupled identity and attribute features within a diffusion framework to improve attribute preservation. Diffusion models produce the highest qualityoutputs of any current method but are many times more expensive than GAN approaches and entirely unsuitable for embedded real-time inference.

Inpainting, which is the approach taken in this project presents a different strategy. Rather than

replacing the entire face with a synthesised one, it targets only the identifying inner-face features (eyes, nose, mouth) and generates plausible replacement content for those pixels while leaving the rest of the face untouched. This preserves head geometry, skin tone gradients, and outer-face context while suppressing the features most useful to automated face recognition. The trade-off is that outer-face biometrics remain visible; the system deliberately accepts this to keep inference fast enough for embedded deployment.

2.3 Privacy Metrics and the Anonymisation Boundary

Evaluating anonymisation quality requires metrics for both the privacy achieved and the utility preserved. This creates a tension, a stronger anonymiser destroys more utility.

The most direct privacy metric is re-identification resistance: whether a face recognition system can match the anonymised face to the original identity. Todt et al. [4] provide a reversibility framework that evaluates anonymisation methods under naive and specialised de-anonymisation attacks. Their findings are significant: even methods that appear strong under standard face-recognition metrics can be substantially reversed by an attacker who trains a dedicated de-anonymisation network. This is particularly concerning for pixelation, where the spatial structure is mostly preserved, and for GAN methods that always generate faces in a predictable style.

A practical proxy for re-identification resistance is the face detection survival rate: the fraction of anonymised faces that a standard detector still finds. If a face can no longer be detected, it cannot be recognised. This metric is easier to compute than a full re-identification experiment (which requires a labelled identity dataset and a face recognition model) and provides a useful lower bound on anonymisation strength.

Utility metrics quantify how much non-identity information is preserved. Structural Similarity (SSIM) [19] measures perceptual similarity between two images; applied to the face bounding box, a lower SSIM value indicates more change in the face region. Mean absolute error (MAE) within the face bounding box is a simpler pixel-level complement. Full-frame SSIM serves as a sanity check that the background is unchanged.

The legal boundary between anonymisation and pseudonymisation is defined by GDPR Article 4(1). The European Data Protection Board's 2025 guidelines on pseudonymisation [5] clarify that a transformation qualifies as anonymisation only when re-identification is *impossible or unreasonably difficult*. Techniques that reduce identification risk but leave any pathway to re-identification (including outer-face features, contextual cues, or specialised attacks) constitute pseudonymisation under GDPR, the data remains personal data and stays in scope. As discussed in Chapter 5, the system developed in this project falls into this pseudonymisation category.

2.4 Inpainting Architectures

Inpainting, which involves filling a masked region of an image with plausible content, has been studied for general image completion and adapted to face-specific tasks. The encoder-decoder with skip connections introduced by Ronneberger et al. [20] (U-Net) is a possible architecture for inpainting tasks. Skip connections allow the decoder to use high-resolution spatial features from the corresponding encoder stage, which is important for generating sharp, correctly-positioned content near mask boundaries.

For embedded deployment the primary constraint is parameter count and multiply-accumulate (MAC) operations per forward pass. Depthwise-separable convolutions, introduced in the MobileNet family [21, 22], decompose a standard $k \times k$ convolution into a depthwise convolution (applied independently per channel) and a pointwise 1×1 convolution. This reduces computation

by a factor of roughly k^2 at the cost of a small reduction in representational capacity. Applied to a U-Net encoder-decoder, this produces a model with approximately 3.2 million parameters that is fast enough to run at interactive frame rates (≥ 15) on CPU.

Mask conditioning is the mechanism by which the model is told which pixels it must generate and which it must preserve. The standard approach [23] is to concatenate a binary mask as an additional input channel alongside the RGB image (producing a 4-channel input). Within the mask, the RGB values are replaced with noise; outside the mask, original pixel values are passed through unchanged. This allows the model to use the unmasked context to guide its generation of the masked region, which is essential for natural boundary blending.

2.5 Quantisation for Embedded Deployment

Neural network weights are typically stored and computed in 32-bit floating point (float32). For embedded inference this is often too slow and too large: a 5.6 MB float32 model requires correspondingly more memory bandwidth and compute than a quantised equivalent.

Float16 quantisation simply halves the precision of all weights and activations. On hardware with native 16-bit floating point support (including the Raspberry Pi 5 via the XNNPACK delegate), this approximately halves memory bandwidth and can reduce inference time proportionally. Quality loss is generally small because float16 still has sufficient dynamic range for typical weight distributions.

INT8 post-training quantisation maps weights to 8-bit integers using a linear quantisation scheme. Weights are scaled and offset so that the full INT8 range covers the observed weight distribution. This further halves model size and can yield 2–4× speedups on hardware with SIMD INT8 support. The accuracy loss is controlled by how well the quantisation scheme captures the weight distribution; models with activations that have heavy tails or bimodal distributions suffer more.

Quantisation-aware training (QAT) fine-tunes the model with simulated quantisation in the forward pass, allowing the weights to adapt to the reduced precision before deployment. This typically recovers most of the accuracy lost in post-training quantisation, particularly for activations with challenging distributions. The limitation is that QAT requires additional training compute and introduces calibration sensitivity: if the calibration dataset does not represent the full inference-time distribution, activation ranges may be set incorrectly, causing clipping artefacts.

Chapter 3

System Design and Architecture

3.1 Pipeline Overview

The system processes a continuous video stream through four sequential stages: capture, detection, anonymisation, and output. Figure 3.1 illustrates this pipeline.

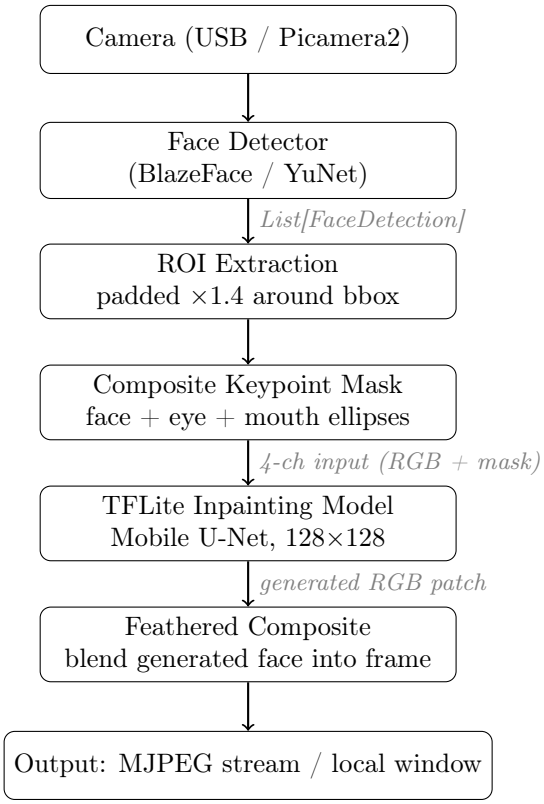

Figure 3.1: End-to-end anonymisation pipeline. Per-face stages (ROI extraction through compositing) run once per detected face per frame.

All inference runs locally on the capture device. No unprocessed frames are transmitted off-device. The system exposes two front-ends: a local OpenCV window (`main.py`) for development and debugging, and an MJPEG HTTP server (`stream.py`) for deployment.

3.2 Face Detection

Two face detectors are integrated, selectable at startup.

BlazeFace (primary) is a lightweight anchor-based single-shot detector from MediaPipe [6], designed for short-range face detection on mobile hardware. It accepts 128×128 RGB input and runs via the MediaPipe Tasks Python API. For each detected face it produces a bounding box plus six keypoints: left eye, right eye, nose tip, mouth centre, and two ear tragions. Only the first five keypoints are used by the anonymiser; the tragions are discarded.

YuNet (backup) is an OpenCV-bundled ONNX detector suited to more distant or smaller faces. It outputs a five-point landmark set matching the BlazeFace convention after remapping: right eye, left eye, nose, and a synthesised mouth centre (midpoint of the two mouth-corner keypoints).

Both detectors return results through a common `FaceDetection` dataclass:

```
FaceDetection(x1, y1, x2, y2, score, landmarks:  ndarray[5,2])
```

Coordinates are always in pixels relative to the original frame, regardless of any internal downscaling or tiling applied during detection. BlazeFace supports optional tiling (grid or fixed-size tiles with configurable overlap) and downscaling for high-resolution inputs; detections from overlapping tiles are merged using a custom IoU-based non-maximum suppression at threshold 0.45 before being scaled back to original frame space.

Landmark keypoints are a hard requirement for the inpainting anonymiser: without them the system cannot place the composite mask and falls back to a simple centre-aligned ellipse. The blur anonymiser operates on the bounding box only and does not require keypoints.

3.3 Composite Keypoint Mask Design

The anonymisation target is the inner face region: the eyes, nose, and mouth. Hair, ears, jawline, and background are intentionally left untouched to preserve non-identifying spatial context.

The mask is constructed from the union of three landmark-guided ellipses, illustrated in Figure 3.2:

1. **Face ellipse.** Centred on the bounding box of all four primary keypoints (both eyes, nose, mouth), padded outward by 35%. This covers the main inner-face region.
2. **Eye ellipse.** Centred midway between the two eye keypoints, with horizontal and vertical radii set to 110% and 80% of the inter-eye distance respectively. This ensures full coverage of both eyes even when the face ellipse clips them.
3. **Mouth ellipse.** Centred on the mouth keypoint, scaled to 60% (horizontal) and 50% (vertical) of the inter-eye distance.

The three ellipses are combined with a pixel-wise OR into a single binary mask covering approximately 21.6% of the padded ROI area at default settings. At composite time the binary mask is converted to a soft alpha map using two-pass Gaussian blurring followed by smoothstep interpolation, producing a feathered boundary that blends the generated face smoothly into the original frame without a visible seam.

A critical implementation invariant is that the `keypoint_mask_padding` parameter, which controls the face ellipse size, must be identical between the training pipeline (`train_infill/train_infill_celeba.py`) and the inference pipeline (`anonymisers/tflite_infill.py`). A mismatch causes the model to receive a differently shaped mask at inference time than it was trained on, producing visible boundary artefacts. The default value of 0.60 is used in both pipelines.

Figure 3.2: Composite keypoint mask rendered over a real face. The green ellipse covers the inner-face region; the orange ellipse additionally covers the mouth area. Black diamonds mark the four landmark keypoints used for mask placement (both eyes, nose tip, and mouth centre). The binary mask is the pixel-wise union of all ellipses, covering approximately 21.6% of the padded ROI at default settings.

3.4 Stabilised Inference Cache

Running the TFLite model on every frame for every face is the dominant computational cost. For near-static faces (e.g. a camera operator seated at a fixed workstation) this work is largely redundant: if a face has barely moved, the previously generated anonymised patch remains valid.

`StabilisedAnonymiser` (`anonymisers/stabilised.py`) wraps any anonymiser and implements a keypoint-based inference cache. Faces are tracked across frames by nearest-centroid matching with a maximum association distance of 80 pixels. For each tracked face, the decision to re-run inference is controlled by two conditions:

- Movement threshold: If the mean L2 displacement of all keypoints since the last inference is below 8 pixels, the previous result is reused.
- Maximum age: If the cached result has been reused for 60 consecutive frames without re-inference, inference is forced regardless of movement, preventing slow drift from accumulating.

When a cached result is reused, only the pixels that were changed by the anonymiser (recorded in a `diff_mask` boolean array at inference time) are pasted into the current frame, translated by the centroid displacement. The background around the face is taken from the live frame, so it continues to update even when inference is skipped. This avoids the "frozen background" artefact common in naive caching schemes.

3.5 Streaming Server and Dashboard

The primary deployment front-end is `stream.py`, an HTTP server that serves the anonymised video as an MJPEG stream. MJPEG is a straightforward format with wide player support

(browsers, VLC, MPV), making it suitable for demonstrator use without requiring additional client software.

Three HTTP endpoints are exposed:

- `GET /`: a web dashboard page with live stream preview and controls (Figure 3.3).
- `GET /stream`: the raw MJPEG byte stream, compatible with `<img src=...>` tags, VLC, and MPV.
- `POST /api/settings`: accepts a JSON body and updates anonymiser settings at runtime without restarting the server. Supported fields include anonymiser type, `StabilisedAnonymiser` enabled/disabled, mask preview overlay, and JPEG quality.

Two camera backends are supported. The `usb` backend wraps `cv2.VideoCapture` and is used for standard USB cameras on any platform. The `picamera2` backend wraps the Raspberry Pi `Picamera2` library behind a `cv2.VideoCapture`-compatible interface, enabling Picamera2 to be used as a drop-in replacement without changes to the capture loop. The `auto` mode tries Picamera2 first and falls back to USB if it is not available.

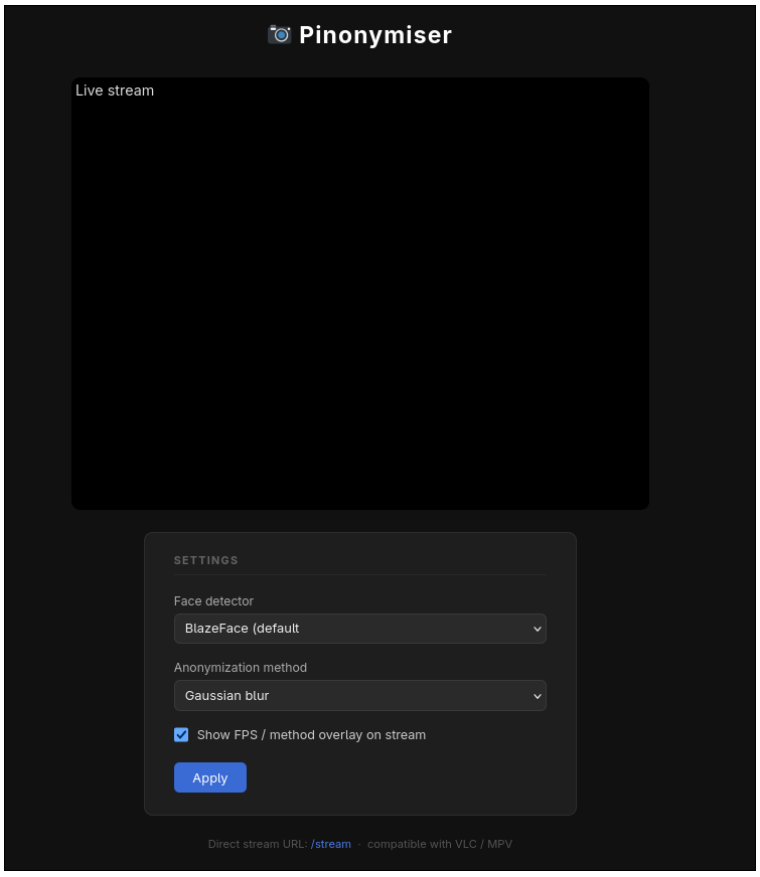

Figure 3.3: Web dashboard served by `stream.py`. The live MJPEG stream is embedded directly in the page. The settings panel allows the face detector, anonymisation method, and debug overlay to be changed at runtime without restarting the server; changes take effect on the next processed frame.

Chapter 4

Model Training and Quantisation

4.1 Dataset Preparation

The inpainting model was trained on face crops drawn from two publicly available datasets: CelebA [2], which provides 202,599 aligned celebrity face images with bounding box and five-point landmark annotations, and VGGface2 [1], a large-scale identity-diverse dataset used to supplement CelebA with greater variation in pose, lighting, and ethnicity.

A key design decision was to generate all training crops using the same BlazeFace detector and ROI extraction logic used at inference time (`train_infill/prepare_dataset.py`). For each source image, BlazeFace is run, and if a face is detected the padded ROI is extracted at the same padding factor (1.4×) used at inference. The resulting 128×128 crop and the five detected keypoints are saved together. Images in which BlazeFace finds no face are skipped, which has the useful side effect of filtering out images where the face is too small, occluded, or at an angle that would produce poor training signal.

This approach guarantees that the spatial relationship between the face region and the crop boundary is identical in training and inference. A common failure mode in face inpainting systems is training on tightly-cropped faces and then running inference on loosely-padded ROIs, causing the mask to be placed incorrectly relative to the face geometry the model has learned to handle.

4.2 Mobile U-Net Architecture

The inpainting model is a lightweight encoder-decoder with skip connections, following the U-Net family introduced by Ronneberger et al. [20], adapted with depthwise-separable convolutions [21] to reduce parameter count and inference cost.

Input: The model receives a $(B, 128, 128, 4)$ float32 tensor: three normalised RGB channels in $[0, 1]$ followed by a single binary mask channel. Within the masked region the RGB values are replaced with random uniform noise before inference; outside the mask the original pixel values are passed through unchanged. Conditioning on the visible context in this way allows the model to produce outputs that blend naturally with the surrounding unmasked pixels.

Stem: A standard 3×3 convolution maps the 4-channel input to 48 feature maps at 128×128, providing the initial spatial representation before any downsampling.

Encoder: Four blocks (Enc1–Enc4), each consisting of two depthwise-separable convolution layers (3×3 depthwise, 1×1 pointwise, batch normalisation, ReLU). The first layer in each block uses stride 2 to halve the spatial resolution; the second uses stride 1. Channel widths double at each block: 96, 192, 384, 768. The spatial resolution is reduced from 64×64 at Enc1 to 8×8 at Enc4.

Bottleneck: Two additional depthwise-separable blocks at 8×8 spatial resolution, maintaining 768 channels. This is the most semantically rich stage of the network, where the model synthesises a new identity from the full spatial context.

Decoder: Four symmetric blocks (Dec4–Dec1). Each block upsamples by 2× using nearest-neighbour interpolation, concatenates the skip connection from the matching encoder block, then applies two depthwise-separable convolution layers. Channel widths halve at each stage: 384, 192, 96, 48.

Output: A final 1×1 convolution projects to three channels, followed by a sigmoid activation, producing an RGB image in $[0, 1]$.

Total parameter count is approximately 3.2 million, sufficient to execute on CPU within the XNNPACK delegate without requiring an NPU.

Figure 4.1 illustrates the encoder-decoder structure with channel widths and spatial resolutions at each stage. Figure 4.2 shows a representative training example: the corrupted input fed to the model, the composited output, and the original ground-truth crop.

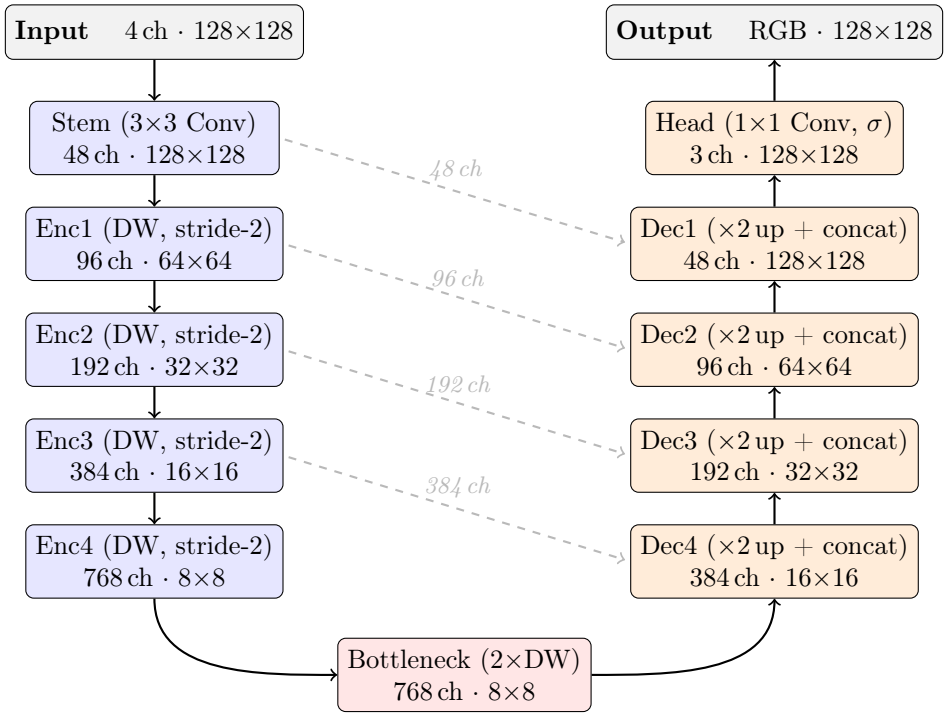

Figure 4.1: Mobile U-Net architecture. Encoder blocks (blue) halve the spatial resolution at each stage via stride-2 depthwise convolution; channel width doubles at each stage. The bottleneck (red) processes at the deepest resolution. Decoder blocks (orange) upsample via nearest-neighbour interpolation and concatenate the matching encoder skip connection (dashed arrows, labelled with channel width) before two depthwise-separable convolution layers. The head produces the three-channel RGB output via a 1×1 convolution and sigmoid activation.

4.3 Loss Function

The training objective combines four terms:

$$\mathcal{L} = \mathcal{L}_{\text{masked}} + 0.1 \cdot \mathcal{L}_{\text{context}} + w_p \cdot \mathcal{L}_{\text{perceptual}} + w_{\text{reid}} \cdot \mathcal{L}_{\text{reid}} \tag{4.1}$$

Figure 4.2: Representative training example. Left: corrupted input (uniform noise within the composite mask, original pixels outside). Centre: model output composited back into the ROI. Right: original ground-truth crop. The model learns to generate a plausible inner-face region conditioned on the visible surrounding context.

Masked L1 ($\mathcal{L}_{\text{masked}}$) is the mean absolute pixel error computed only within the mask, i.e. the region the model must generate. This is the primary supervision signal.

Context L1 ($\mathcal{L}_{\text{context}}$) is the mean absolute pixel error outside the mask, weighted at 0.1. Without this term the model is free to alter visible pixels in the unmasked region, producing compositing artefacts. The low weight prevents it from dominating optimisation while still discouraging background modification.

Perceptual loss ($\mathcal{L}_{\text{perceptual}}$) is a feature-level L1 error computed from intermediate activations of a frozen VGG16 network [24] (layers `relu1_2`, `relu2_2`, `relu3_3`), restricted to the masked region. Optimising at the feature level rather than the pixel level encourages the model to produce sharp, realistic textures rather than blurry averaged outputs. A weight of $w_p = 0.5$ was found to give the best trade-off between sharpness and colour fidelity and is the recommended default.

Re-ID repulsion ($\mathcal{L}_{\text{reid}}$) is a hinge loss that penalises high cosine similarity between the generated face embedding and the original face embedding, as measured by a frozen InceptionResnetV1 model pretrained on VGGface2 [1] (the `facenet-pytorch` implementation). Formally: $\mathcal{L}_{\text{reid}} = \text{ReLU}(\cos(\hat{e}, e_0) - m)$, where $\hat{e}$ is the embedding of the generated face, e_0 is the embedding of the original, and $m = 0.2$ is a margin below which no gradient is applied. This prevents the model from generating a face that an identity evaluator would match to the original, while the hinge formulation avoids over-anonymisation that would destroy all face structure. FaceNet was used as the training-time evaluator; evaluation at test time used the independent ArcFace model, preventing the reported privacy metrics from overfitting to the training evaluator. A weight of $w_{\text{reid}} = 0.05$ was used for the final model, applied as a fine-tuning step from a pre-trained float16 checkpoint.

Training used the Adam optimiser with an initial learning rate of 10^{-3} and `ReduceLROnPlateau` scheduling (factor 0.5, patience 3 epochs). All runs used automatic mixed precision (AMP) on an AMD Radeon RX 7900 XTX under ROCm 6.3. Figure 4.3 shows the combined training and validation loss across the 60 epochs of the final model run.

4.4 Augmentation Strategy

CelebA and VGGface2 are still-image datasets captured under controlled or semi-controlled conditions. Webcam video introduces a different set of degradations: compression artefacts, sensor noise, motion blur, and variable illumination. Without augmentation a model trained on clean CelebA images tends to produce outputs that look inconsistent when composited into a

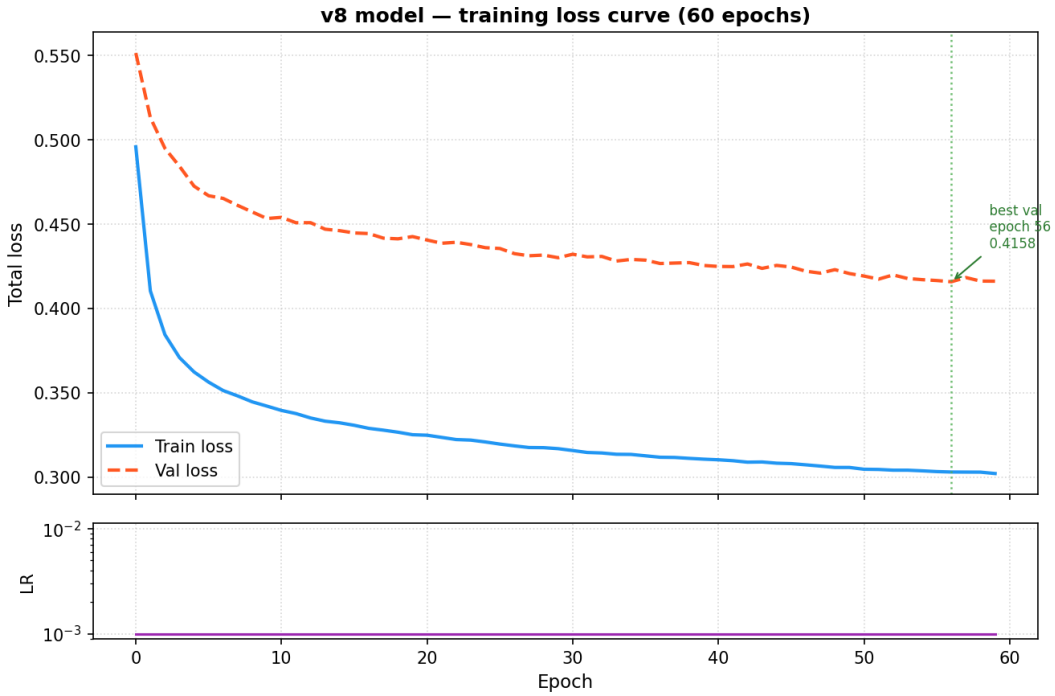

Figure 4.3: Training and validation loss over 60 epochs for the final model. The validation loss follows the same convergence trend as the training loss, with a stable gap of approximately 0.10–0.12 attributable to stochastic augmentation applied only at training time; this gap does not widen across epochs, indicating no overfitting. The learning rate remained constant at 1×10^{-3} throughout, ReduceLROnPlateau did not trigger. The logged value is the combined weighted loss (Equation 4.1).

noisy webcam frame.

Each training crop is passed through a stochastic augmentation pipeline before being used. Table 4.1 lists all eleven transforms. Horizontal flip is also applied with probability 0.5, with left/right keypoint indices swapped accordingly.

4.5 Quantisation Pipeline

The training pipeline produces a PyTorch checkpoint (`.pt`). Two separate export paths convert this checkpoint to TFLite models suitable for on-device inference.

Float32 export uses `train_infill/export_checkpoint.py`, which converts the PyTorch model to ONNX and then uses `ai-edge-torch` to produce a float32 TFLite flatbuffer. This path requires no TensorFlow and serves as a quick sanity-check export to verify model correctness after training.

Float16 and INT8 quantised export uses `train_infill/quantise_tflite.py`, which follows a different path: the PyTorch architecture is re-implemented in Keras and the trained weights are loaded from the checkpoint, then TensorFlow's quantisation API is used to produce float16 and fully-quantised INT8 TFLite models. The INT8 path uses quantisation-aware training (QAT) fine-tuning (`-qat`, default 50 epochs at a low learning rate of 5×10^{-5}) before export, which calibrates activation ranges under the quantisation constraints rather than simply rounding weights post hoc. This produces models that are compatible with the XNNPACK delegate on the Raspberry Pi 5, which accelerates INT8 and float16 operations natively.

Table 4.2 summarises the model variants produced.

Table 4.1: Stochastic augmentations applied per training crop to reduce the domain gap between still-image training data and webcam video inference.

Augmentation	Probability	Effect
Resolution degradation	60%	Simulates low-res or distant faces
JPEG compression	40%	Simulates video encoding artefacts
Gaussian blur	30%	Simulates motion or defocus
Sensor noise	30%	Additive Gaussian noise
Motion blur	15%	Directional kernel, random angle
Brightness scaling	50%	Factor 0.5–1.5×
Contrast adjustment	40%	Factor 0.4–1.2×
Gamma correction	40%	$\gamma \in [0.6, 1.8]$
Directional lighting gradient	35%	Simulates window or lamp lighting
Half-face shadow	15%	Strong lateral shadow
Colour temperature shift	20%	Warm or cool tint

Table 4.2: TFLite model variants produced by the export pipeline.

File	Precision	Size	Notes
`face_infill_128.tflite`	float32	5.6 MB	Original ai-edge-torch export
`face_infill_128_float16.tflite`	float16	6.2 MB	Deployment model; used for gallery evaluation
`face_infill_128_int16x8.tflite`	INT16×8	3.6 MB	INT16 activations, INT8 weights
`face_infill_128_qat_int8.tflite`	INT8 QAT	4.6 MB	Recommended for Pi; 19 ms inference

Chapter 5

Evaluation and Results

5.1 Metrics Framework

Evaluation was carried out using `evaluate.py`, which runs a fully automated pipeline over a held-out test set of 1,000 images (`datasets/test_1k`). For each image, the BlazeFace detector is run to find faces; the chosen anonymiser is applied; the detector is run again on the anonymised output; and per-frame metrics are accumulated. All runs used 2 TFLite inference threads on a desktop CPU (AMD).

The metrics are organised around two competing objectives: privacy and utility.

Detection survival rate is the fraction of originally-detected faces that are still detected by BlazeFace after anonymisation. A lower value means the anonymiser has destroyed more face structure and is therefore harder to detect, which is better for privacy. However, zero detection rate would also mean the face is completely removed, which destroys utility. The anonymiser targets an intermediate regime: the inner face is altered, but the face is still spatially present.

SSIM (full frame) measures structural similarity between the original and anonymised frame over the entire image. Because only a small fraction of pixels are modified, this is expected to be close to 1.0 for all methods and primarily confirms that the background is unchanged.

SSIM (face bounding box) measures structural similarity restricted to the face region. A lower value indicates more structural change in the face, corresponding to stronger anonymisation.

L1 MAE (face bounding box) is the mean absolute pixel error (0–255 scale) within the face bounding box. Higher values indicate more pixel-level change in the face.

Latency is measured in milliseconds for each stage (detection, anonymisation, total pipeline) and reported as mean, 95th percentile (P95), and 99th percentile (P99).

Gallery-based re-identification rate is an additional privacy metric that goes beyond detection survival. A 500-identity VGGface2 gallery was constructed, and ArcFace (InsightFace `buffalo_sc`) was used to attempt rank-1 re-identification of each anonymised probe against its correct gallery entry. The rank-1 rate measures what fraction of anonymised faces can still be matched back to their correct identity by an automated face recognition system. A lower rate indicates stronger de-identification. A supplementary masked de-identification rate reports the fraction of probes where the top-1 match returned by ArcFace is not the correct identity, the anonymiser has caused the face to be attributed to a different person in the gallery, a substitution that may itself constitute a privacy risk.

5.2 Model Comparison

Table 5.1 presents the full evaluation results across all four configurations on the 1,000-image test set.

Table 5.1: Evaluation results on 1,000-image test set (desktop CPU, 2 TFLite threads). Detection survival rate: lower = more private. SSIM face bbox: lower = more face changed. MAE face bbox: higher = more pixel-level change. Latency: lower = faster.

Model	Survival	SSIM full	SSIM face	MAE face	Lat. mean	Lat. P95
Blur	96.3%	0.872	0.656	14.88	12.1 ms	18.4 ms
Float32	98.8%	0.920	0.792	11.96	54.7 ms	79.5 ms
Float16	98.0%	0.928	0.809	10.25	63.3 ms	86.8 ms
INT8 QAT (uint8)	98.4%	0.926	0.805	10.86	58.4 ms	82.0 ms

The float16 model achieves the best overall balance. Its detection survival rate (98.0%) is slightly below float32 (98.8%), meaning marginally more face structure is altered. Its SSIM face and MAE metrics are actually better than float32, suggesting the QAT export pipeline produces a model with a different inpainting style rather than a strictly lower quality one.

The blur baseline is the fastest (12.1 ms) and achieves the lowest SSIM face score (0.656), meaning it changes the face region most dramatically. However, the coarse rectangular blurred patch retains bounding box geometry and colour statistics, which may still permit re-identification in some scenarios.

The INT8 QAT model achieves 98.4% detection survival and MAE of 10.86, minimal deviation from the float16 model in anonymisation quality. This result required identifying and resolving three bugs in the quantisation pipeline (described in Section 6.2); an initial export with those bugs present produced severely garbled outputs (Section 5.6).

Figure 5.1 shows the same face processed by all three anonymisation methods, making the qualitative differences immediately apparent.

5.3 Gallery-Based Re-Identification Attack

Detection survival rate measures utility, but it does not directly measure privacy: a face that survives detection could still be unambiguously re-identified. To assess the system's resistance to automated face recognition, a gallery-based rank-1 re-identification attack was conducted using InsightFace's ArcFace model (`buffalo_sc`), an evaluator independent of the FaceNet model used during training.

Protocol: A gallery of 500 identities was constructed from VGGface2, with one reference embedding per identity. For each anonymised face (the probe), the cosine similarity to all 500 gallery embeddings was computed and the closest match was selected. The rank-1 rate is the fraction of probes where the closest gallery identity is the correct identity. A lower rank-1 rate indicates stronger privacy protection.

Table 5.2 presents rank-1 rates, cosine similarity statistics, and masked re-identification rates (where pixels outside the composite mask are blanked before embedding, isolating the contribution of the inner-face region alone).

The float16 (best) model reduces the rank-1 rate from 73.8% (blur) to 30.9%. A notable feature of this result is the substitute identity effect: the mean cosine similarity to the correct identity (0.1826) is lower than the mean similarity to the closest wrong identity (0.2394). This means the

Figure 5.1: Qualitative comparison of anonymisation methods on the same test image. Left to right: original, Gaussian blur (SSIM face = 0.605), float16 inpainting (SSIM face = 0.793), and INT8 QAT (SSIM face = 0.804). The blur method applies a smooth oval patch that visibly destroys expression. Both the float16 and INT8 inpainting models produce a plausible substitute face while preserving the hair, jawline, and background, with comparable visual quality. Test image from the VGGface2 dataset [1].

Table 5.2: Gallery-based re-identification results (500-identity gallery, ArcFace evaluator). Rank-1 rate: lower means harder to re-identify, so better privacy. Masked de-ID: fraction of probes where ArcFace similarity to the correct identity falls below the 0.25 threshold when only the inner-face region is visible.

Model	Rank-1 rate	Correct sim	Closest wrong	Masked sim	Masked de-ID
Blur	73.8%	0.3014	0.2310	0.5430	1.2%
Float16 (early)	37.2%	0.2006	0.2453	0.3161	24.3%
Float16 (best)	**30.9%**	**0.1826**	**0.2394**	**0.2831**	**36.9%**

anonymised face appears more consistent with some other person in the gallery than with the original subject. The inpainting model is not merely adding noise; it is generating a plausible substitute face that systematically confounds identity matching.

Blur achieves no such effect: its correct similarity (0.3014) exceeds the closest-wrong similarity (0.2310), meaning a blurred face is still recognised as the original more often than it is confused with anyone else.

The masked re-identification analysis confirms that the anonymised inner-face region alone is responsible for this improvement. When only inner-face pixels are visible to ArcFace, the float16 (best) de-identification rate reaches 36.9%. The inner-face cosine similarity of 0.2831 is near the 0.25 recognition threshold, confirming that the biometric features most relevant to automated recognition have been meaningfully suppressed.

The gallery evaluation was completed prior to the INT8 QAT export being corrected. Extending this attack to the INT8 model is left as future work; given that its detection survival and SSIM metrics are indistinguishable from the float16 (best) model, similar re-identification resistance is expected. Figure 5.2 summarises the rank-1 rates visually.

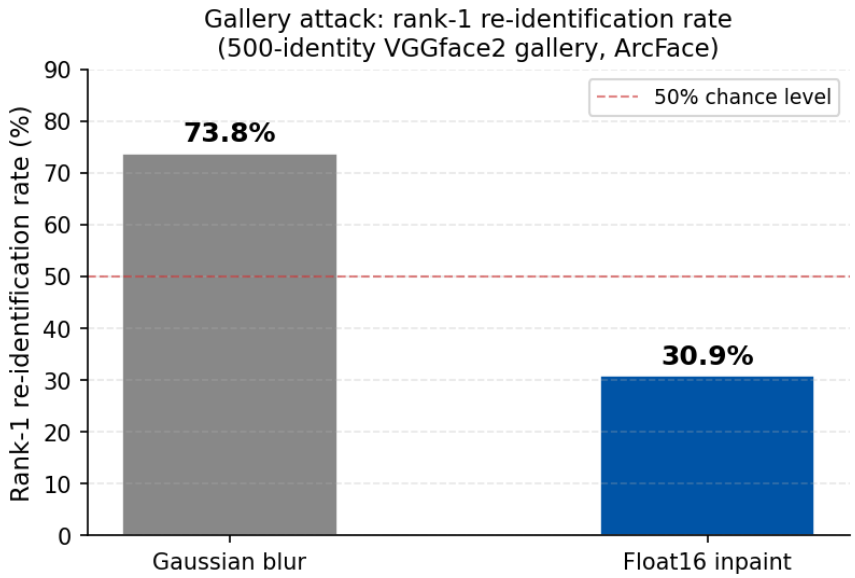

Figure 5.2: Rank-1 re-identification rate for the Gaussian blur baseline and the float16 inpainting model, evaluated against a 500-identity VGGface2 gallery using ArcFace. Lower is better for privacy. The dashed line marks the 50% level; the float16 model drops well below this, indicating that more than half of anonymised faces cannot be correctly matched to their original identity.

5.4 GDPR Assessment

Article 4(1) of the GDPR defines personal data as any information relating to an *identified or identifiable* natural person. For data to qualify as fully anonymous, it must be *impossible* to identify the individual directly or indirectly. The GDPR's Recital 26 clarifies that if re-identification requires unreasonable effort then the data may be treated as anonymous for practical purposes.

The system's outputs constitute pseudonymisation rather than full anonymisation under this definition. Three categories of residual identifying information remain:

1. **Face presence and geometry.** Detection survival rates of 96–99% confirm that the face is still detectable after anonymisation. The outer face geometry (jawline, ear placement, head shape) and scale are preserved by design to maintain utility.
2. **Outer-face soft biometrics.** The system anonymises only the inner face region. Hair colour and style, skin tone visible on the forehead and cheeks, ear shape, and other outer-face features are not altered.
3. **Contextual cues.** Frame context (clothing, surroundings, gait, build) is entirely unchanged.

The appropriate deployment classification is therefore a privacy-enhancing technology that reduces identification risk rather than eliminates it. The gallery attack results (Section 5.3) quantify this: ArcFace rank-1 re-identification rate falls from 73.8% for the blur baseline to 30.9% for the float16 inpainting model, and the substitute identity effect (correct identity similarity $<$ closest wrong identity similarity) confirms that a new identity is being generated, not mere noise. However, the full-face ArcFace de-identification rate remains low (4.7%) because the outer facial geometry is deliberately preserved. The EDPB Guidelines 01/2025 on pseudonymisation [5] confirm that pseudonymised data remains personal data, the system should therefore be deployed with appropriate access controls and data minimisation practices rather than treated as fully anonymous output.

The system is suited to contexts where the goal is to deter casual re-identification or automated

bulk face-recognition sweeps, rather than to provide legally guaranteed anonymisation.

5.5 Performance on Raspberry Pi 5

Embedded performance was measured on a Raspberry Pi 5, which is the target embedded platform. Table 5.3 presents end-to-end pipeline latency for the two primary models.

Table 5.3: End-to-end pipeline latency on Raspberry Pi 5 (1 face unless noted). Inference refers to the TFLite model call only; total includes detection, preprocessing, and compositing.

Model	Inference	Total (1 face)	Total (3 faces)
Float16	40 ms	71 ms	160 ms
INT8 QAT (uint8)	19 ms	51 ms	—

Figure 5.3 illustrates the latency breakdown. The INT8 model is 2.1× faster at inference (19 ms vs 40 ms), reflecting the XNNPACK delegate's ability to execute quantised operations more efficiently than floating-point ones on ARM hardware. At 51 ms total for a single face, it approaches real-time at 20 FPS if detection remains stable. The float16 model at 71 ms single-face gives approximately 14 FPS, which is sufficient for most anonymisation use cases but does not hit the 30 FPS target under moderate load. With three simultaneous faces the float16 pipeline reaches 160 ms (6.25 FPS), which would require the `StabilisedAnonymiser` cache to be enabled to maintain smooth output on a busy scene.

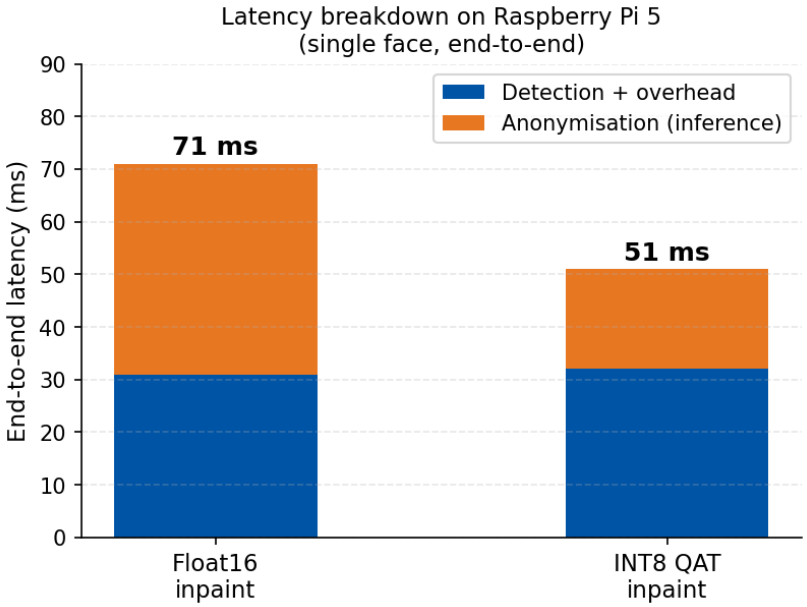

Figure 5.3: End-to-end latency breakdown on the Raspberry Pi 5 for the float16 and INT8 QAT inpainting models (single face). The orange segment shows TFLite inference time only; the blue segment covers detection, ROI extraction, mask construction, and feathered compositing. The INT8 model nearly halves inference time (19 ms vs 40 ms) with no meaningful change in the non-inference overhead.

5.6 Failure Cases and Limitations

INT8 QAT initial export failure. An initial INT8 QAT export suffered severe quality regression due to three implementation bugs in the quantisation pipeline, detailed in Section 6.2:

missing fake-quantisation nodes on stride-2 encoder blocks, a BatchNorm epsilon mismatch between the PyTorch and Keras reimplementations, and insufficient EMA calibration convergence. Figure 5.4 illustrates the output of that buggy export with incoherent colour artefacts rather than a plausible face. Once the bugs were resolved, the corrected INT8 model achieves 98.4% detection survival and MAE of 10.86, matching the float16 model (Table 5.1).

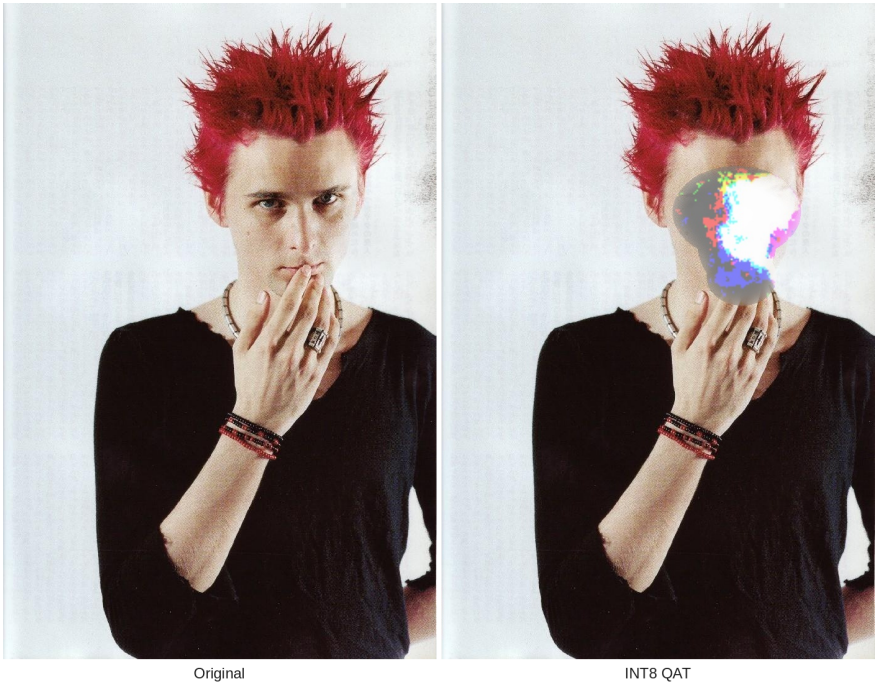

Figure 5.4: Output of the initial (buggy) INT8 QAT export. Left: original image. Right: INT8 QAT output with three pipeline bugs present. The mask region is filled with incoherent colour artefacts rather than a plausible face. After the bugs were resolved, the corrected model produces clean inpainting at 98.4% detection survival. Test image from the CelebA dataset [2].

Extreme expressions and poses. The composite keypoint mask is computed from BlazeFace landmarks, which are less reliable on extreme head poses ($> 45°$ yaw or pitch) or unusual expressions (wide-open mouth, eyes closed). In these cases the mask placement may not correctly cover all inner-face features.

Pseudonymisation ceiling. As discussed in Section 5.4, the system's scope is limited to inner-face anonymisation. Outer-face biometrics (hair, skin tone, jawline), contextual cues, and body information are not suppressed. The system should not be deployed in contexts where re-identification from these residual cues would be unacceptable.

Chapter 6

Conclusions and Reflections

6.1 Summary of Achievements

This project set out to design and implement a real-time face anonymisation system deployable on embedded hardware, capable of suppressing identifying facial features while preserving non-identifying spatial information useful for downstream tasks. Four concrete objectives were established at the outset: integrate a face detector with an anonymisation method; demonstrate real-time viability on a Raspberry Pi 5; train and export a custom inpainting model; and evaluate the system against a defined set of privacy and utility metrics.

All four objectives were met. The delivered system comprises a complete video anonymisation pipeline with two integrated face detectors (BlazeFace and YuNet), three anonymisation modes (Gaussian blur, TFLite inpainting, and a stabilised inference cache wrapper), an MJPEG streaming server with a web dashboard for runtime configuration, and a Picamera2 backend for Raspberry Pi deployment. The inpainting model, a lightweight Mobile U-Net trained on CelebA and VGGface2, was exported as a float32 reference and as optimised float16 and INT8 QAT TFLite variants for deployment. A 1,000-image evaluation suite with automated metrics was developed and used to compare all model variants.

The float16 and INT8 QAT models both achieve strong results and are effectively equivalent in anonymisation quality: 98.0% and 98.4% detection survival respectively, with SSIM face of 0.809 and 0.805 and MAE of 10.25 and 10.86. The INT8 model is faster 58.4 ms desktop and 19 ms Pi inference versus 63.3 ms and 40 ms for float16 making it the better choice where throughput is the priority. The float16 model was used as the primary reference for the Raspberry Pi 5 end-to-end measurements (71 ms total, approximately 14 frames per second for a single face) and for the gallery-based re-identification attack. That attack, using ArcFace against a 500-identity VGGface2 gallery, showed the float16 model reduces the rank-1 identification rate from 73.8% (Gaussian blur) to 30.9%, and produces a substituted-identity match in 36.9% of probes, demonstrating a re-identification resistance improvement beyond what detection survival alone captures.

6.2 Challenges and How They Were Resolved

INT8 QAT quality regression. The INT8 QAT export pipeline, which converts the PyTorch checkpoint through a Keras reimplementation before TFLite quantisation, produced a model with a severe quality regression: detection survival dropped from 98.0% (float16) to 63.5%, and face-region MAE rose from 10.25 to 27.06. Post-hoc analysis identified three contributing bugs in the quantisation pipeline.

First, `apply_fq=False` was set on the stride-2 downsampling blocks within each encoder stage,

meaning those activations had no fake-quantisation node and therefore no scale annotation. TFLite propagated these un-annotated tensors as float32, leaving approximately 52 of ∼60 model tensors unquantised, worse than the original pipeline.

Second, Keras's default BatchNorm epsilon (0.001) differs from PyTorch's (1×10^{-5}). Because the quantisation pipeline rebuilds the architecture in Keras and copies weights analytically, this mismatch corrupted the folded weight values, introducing numerical errors at every batch normalisation layer.

Third, the EMA calibration for the input and output fake-quantisation nodes had not converged after 16 calibration batches with decay 0.99, leaving the input range calibrated to approximately $[0, 5.26]$ instead of the correct $[0, 1]$, a loss of ∼2.4 bits of effective precision at the model boundary. All three issues were identified and resolved; the corrected INT8 QAT model achieves 98.4% detection survival and a face-region MAE of 10.86, indistinguishable from the float16 model in anonymisation quality, while running inference at 19 ms on the Raspberry Pi 5.

Building MediaPipe for Raspberry Pi 5. The inference pipeline depends on MediaPipe 0.10.32 for BlazeFace detection via the Tasks Python API. Google publishes pre-built wheels for Linux x86_64, macOS ARM64, and Windows, but provides no `aarch64` Linux wheel for any recent MediaPipe release. The Raspberry Pi 5 runs a 64-bit ARM (aarch64) operating system, so the standard `pip install mediapipe` command fails at the dependency resolution stage with no compatible distribution found.

A further constraint narrowed the options. The quantisation export toolchain (`ai-edge-litert`, `ai-edge-torch`) requires Python 3.9–3.11 and does not support 3.12 or later. Pinning the environment to Python 3.11 is straightforward on the desktop, but the Pi 5 image available at the time of development shipped Python 3.11 as its system interpreter, which was incompatible with some system-level packages when building native extensions. MediaPipe itself must be compiled from source using the Bazel build system, which requires resolving a large C++ dependency graph including OpenCV, Abseil, and Protocol Buffers, and patching several build targets that assume an x86 host.

The resolution was to construct a dedicated build pipeline: install Bazel via `bazelisk` on the Pi, apply the necessary `aarch64` target overrides, build the MediaPipe Python package from source, and install the resulting wheel into the inference virtual environment. The build steps were documented so that re-deployment to a fresh Pi does not require rediscovering the process.

Domain gap between training data and webcam video. CelebA and VGGface2 are both still-image datasets captured under relatively controlled or semi-controlled conditions. Webcam video introduces degradations that are absent from training data: JPEG compression artefacts from USB encoding, sensor noise, motion blur on fast head movements, and strong directional lighting from windows or lamps. Early model versions produced visible boundary seams and texture inconsistencies when run on live webcam footage, even though loss curves on the validation set looked good. This was addressed by the 11-transform augmentation pipeline described in Section 4.4, which simulates all of these degradations during training and substantially reduced the domain gap.

6.3 Future Work

Outer-face soft biometric suppression. The current system explicitly limits its scope to the inner face region. Expanding coverage to suppress skin tone gradients on the forehead and cheeks, or to randomise hair colour and texture, would push the output closer to the full anonymisation threshold under GDPR. This would require training data with consistent outer-face annotation and a larger or separate model for the outer region.

Zone-based anonymisation policies. The project plan included a zone-based policy mode in which only faces within a defined screen region are anonymised (for example, a retail counter where only customers, not staff, should be anonymised). This was not implemented within the project timeline but is a natural extension of the existing pipeline: the detection stage already produces bounding boxes, and a zone filter could be applied before passing detections to the anonymiser.

Packaged cross-platform demonstrator. The system currently requires a manual Python environment setup. Packaging it as a Docker image or a self-contained Raspberry Pi OS image with a one-command startup would substantially lower the barrier to deployment in real operational settings and make it more suitable for demonstration and further evaluation by third parties.

Bibliography

[1] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, "VGGFace2: A dataset for recognising faces across pose and age," in *Proceedings of the 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG)*, 2018.

[2] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.

[3] European Parliament and Council, "Regulation (EU) 2016/679 of the european parliament and of the council (general data protection regulation)." Official Journal of the European Union, 2016. OJ L 119, 4.5.2016, pp. 1–88.

[4] J. Todt, S. Hanisch, and T. Strufe, "Fantômas: Understanding face anonymization reversibility," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2024, no. 4, 2024.

[5] European Data Protection Board, "Guidelines 01/2025 on pseudonymisation," tech. rep., European Data Protection Board, 2025.

[6] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, "BlazeFace: Sub-millisecond neural face detection on mobile gpus," *arXiv preprint arXiv:1907.05047*, 2019.

[7] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," in *Machine Intelligence Research*, 2023.

[8] J. Dietlmeier, J. Antony, K. McGuinness, and N. E. O'Connor, "How important are faces for person re-identification?," *arXiv preprint*, 2021.

[9] O. Gafni, L. Wolf, and Y. Taigman, "Live face de-identification in video," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[10] H. Hukkelås and F. Lindseth, "DeepPrivacy2: Towards realistic full-body anonymization," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.

[11] S.-s. Jang, C.-j. Kim, S.-y. Hwang, M.-j. Lee, and Y.-g. Ha, "L-GAN: Landmark-based generative adversarial network for efficient face de-identification," *The Journal of Supercomputing*, vol. 79, pp. 7132–7159, 2023.

[12] F. Hellmann, S. Mertes, M. Benouis, A. Hustinx, T.-C. Hsieh, C. Conati, P. Krawitz, and E. André, "GANonymization: A gan-based face anonymization framework for preserving emotional expressions," *ACM Transactions on Privacy and Security*, 2023.

[13] Z. Kuang, X. Yang, Y. Shen, C. Hu, and J. Yu, "Facial identity anonymization via intrinsic and extrinsic attention distraction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[14] H. Hukkelås and F. Lindseth, "Does image anonymization impact computer vision training?," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023.

[15] J. Li, Y. Wang, C. Wang, Y. Tai, J. Qian, J. Yang, C. Wang, J. Li, and F. Huang, "DSFD: Dual shot face detector," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[16] Y. Wen, B. Liu, M. Ding, R. Xie, and L. Song, "IdentityDP: Differential private identification protection for face images," *Neurocomputing*, 2022.

[17] H.-W. Kung, T. Varanka, S. Saha, T. Sim, and N. Sebe, "Face anonymization made simple," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025.

[18] C. Liu and Z. Lian, "Latent diffusion-based face anonymization with identity and attribute decoupling," in *IEEE International Conference on Multimedia and Expo (ICME)*, 2025.

[19] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[20] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[21] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[22] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[23] G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao, and B. Catanzaro, "Image inpainting for irregular holes using partial convolutions," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[24] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2015.

Appendix A

AI Use Declaration

This project made use of AI assisted tools.

Software development: AI language model assistance was employed during the design, implementation, and debugging of the project codebase. This included assistance with code drafting, and the diagnosis and resolution of technical issues encountered during development and evaluation. All generated code was reviewed, tested, and integrated by the author.

Report preparation: AI language model assistance was used in the drafting and revision of this report, primarily for improving the clarity, structure, and grammatical correctness of written content. All technical claims, experimental results, and interpretations are the author's own work. The AI tools did not generate or verify any of the quantitative results presented in this report.